深度学习课程设计技术报告

基于 ChnSentiCorp 的中文情感分类对比

学号：52612048025　　姓名：马陈奕

课程：深度学习课程设计（2026-秋季）

代码仓库：https://github.com/machenyi170/dl-course-sentiment

一、研究问题与动机

这门课的结课作业，我选的是中文情感二分类：判断酒店评论是正面还是负面。数据集用 ChnSentiCorp，下载方便，体量小。

做法上是把常见的三类方法放在同一套数据划分里去比较：传统的 TF-IDF + 线性 SVM、BiLSTM，以及比较小的中文预训练模型 hfl/rbt3。统一看 Accuracy 和 Macro-F1，加上了一点简单消融，结束后再抽几条错例去看问题出在哪。代码和结果都放到了仓库里，方便复现。

二、数据与实验环境

2.1 数据集

数据用的是 HuggingFace 上的 lansinuote/ChnSentiCorp，任务是酒店评论二分类。划分用的是官方的：训练集 9600、验证集 1200、测试集 1200；标签 0 表示负面、1 表示正面，两类数量大体均衡。评论长短差得比较多。我去掉了空文本，然后按各方法需要做分词或编码。

2.2 环境与评价指标

实验在 Linux、CPU 环境下完成（Python 3.13，PyTorch CPU 版），主要用了 scikit-learn、jieba、Transformers、datasets，以及画图相关的库。评价指标用的是 Accuracy 和 Macro-F1。神经网络按验证集 Macro-F1 选最好的一轮，再到测试集上看最终结果。随机种子固定为 42。

三、方法

3.1 TF-IDF + Linear SVM

先用 jieba 分词，再做成 TF-IDF（最多 3 万维，1–2 gram，min_df=2），上面接 LinearSVC（C=1.0）。这个方法实现快、跑得也快，当作基线。

3.2 BiLSTM

BiLSTM 按词来建模：词表只保留训练集里出现不少于 2 次的词，句子截断到 128。词向量 100 维，单层双向 LSTM，隐层 128，把两端最后状态拼起来再分类。优化用到了 Adam 和交叉熵，训练 6 个 epoch，并做了梯度裁剪。消融对比了两种设置：词向量跟着一起训练，以及冻住随机初始化的词向量。更大的中文预训练词向量不好被完整下载，所以用冻住 / 不冻住词向量做对比，去看嵌入层有没有用。

3.3 预训练中文编码器 + 分类头

预训练部分用的是比较小的中文模型 hfl/rbt3（3 层，CPU 能跑），最大长度 128，batch size 16。设置了两种：一种是编码器全部冻住，只训练上面的分类头；另一种放开最后两层，用较小的学习率做微调。可以看出来“当特征提取器用”和“稍微改一下适配任务”差多少。

四、设计与实现要点

代码按脚本拆开：prepare_data.py 负责下载和整理数据；train_svm、train_bilstm、train_bert 负责各自训练；plot_results.py 负责出图。指标都写进了 JSON/CSV。三类方法共用同一套官方划分。BiLSTM 用 pack_padded_sequence 处理长短不一的句子；BERT 按验证集 Macro-F1 保存最好的权重。以上脚本都是按 CPU 环境写的，本机也能直接跑。

五、实验结果

验证集和测试集的主要结果见表 1。

表 1　各方法验证/测试性能对比

方法	验证 Accuracy	验证 Macro-F1	测试 Accuracy	测试 Macro-F1
TF-IDF+SVM	90.58%	90.57%	91.50%	91.50%
BiLSTM-随机词向量	86.08%	86.08%	86.67%	86.63%
BiLSTM-冻结词向量	83.50%	83.50%	84.33%	84.30%
BERT-冻结编码器	80.58%	80.55%	79.92%	79.71%
BERT-微调末两层	89.25%	89.25%	90.00%	89.99%

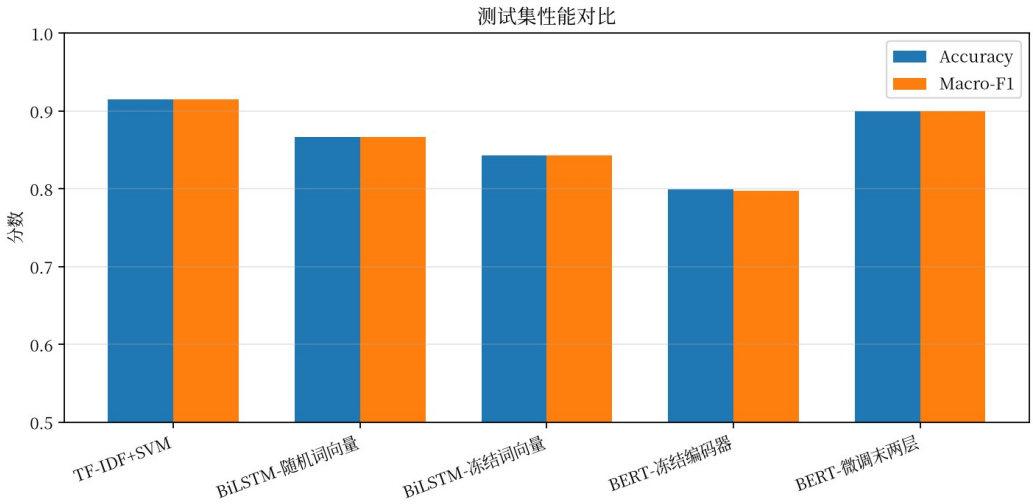


图 1　测试集 Accuracy 与 Macro-F1 对比

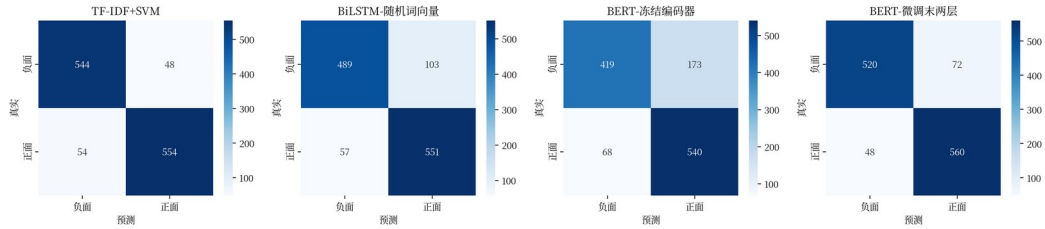


图 2　主要方法测试集混淆矩阵

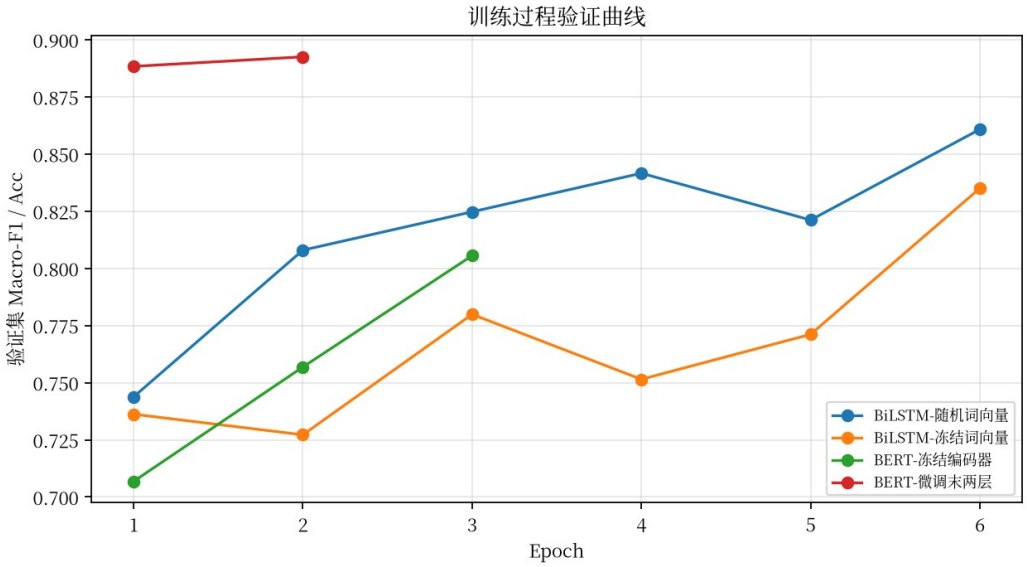


图 3　验证集训练曲线

测试集上 Macro-F1 最高的是 TF-IDF+SVM，准确率和 Macro-F1 都是 91.50%，在这个评论数据上表现很强。BiLSTM（词向量可训练）的 Macro-F1 是 86.63%；把词向量冻住以后降到 84.30%，说明让嵌入跟着学还是有用的。BERT 编码器全冻住时只有 79.71%；放开最后两层微调后升到 89.99%。

六、分析、思考与改进

分析：在酒店评论里，“脏”“差”“满意”“干净”这类情感很分明的词很容易被 TF-IDF 抓住。如果数据量不大、词向量也不强的话，BiLSTM 不一定能超过 SVM 这条基线，所以把强基线留着还是有必要的。消融结果也差不多：BiLSTM 让词向量一起训练，比冻住更好；BERT 微调最后两层，比整段编码器都冻住更好。

思考：我看了测试集上的错例。比如“这个宾馆比较陈旧了……总体来说一般”，标签是正面，模型却判成负面，主要是语句比较平，没有很明显的褒义词。还有“3999 的时候抢购的……品牌和价格都很不错”这种，我们自己能听出来是褒义，但是标签是负面，可能是标注错了，也可能是缺上下文。另外还有一些装系统、改 BIOS 的说明文字混在数据里，跟情感没什么关系，可能也会把模型带偏。词袋和比较浅的序列模型，碰到转折句、标错的样本都比较容易出错。后面如果有算力，可以换更大的中文模型做全参微调，顺便把数据清一清，或者改成方面级情感分析。

局限是：为了能在 CPU 上跑完，我用的是轻量 rbt3，超参没有细搜，也没拿更大的 chinese-roberta-wwm-ext 之类做全量微调。但条件允许的话，这些都可以再补。

七、结论

这次作业把选题、数据、三种方法、对比消融和画图都跑通了。在这个数据集上，TF-IDF+SVM 是最强的；BiLSTM 在词向量一起训练时会好一点；小预训练模型冻住编码器时效果一般，微调最后两层会好很多。

参考文献

[1] 谭松波. ChnSentiCorp 中文情感分析语料. 本实验使用：https://huggingface.co/datasets/lansinuote/ChnSentiCorp

[2] Cui Y, Che W, Liu T, et al. Revisiting Pre-Trained Models for Chinese Natural Language Processing. Findings of EMNLP, 2020. 模型：https://huggingface.co/hfl/rbt3

[3] Pedregosa F, et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.

[4] Hochreiter S, Schmidhuber J. Long Short-Term Memory. Neural Computation, 1997.

[5] Devlin J, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.